

Dict, Tuple, Set

Q1: จงสร้าง dict เพื่อเก็บข้อมูลด้านล่าง

grade_dict

Name	Grade
Alice	A
Bob	B+
John	B+
Doe	F

score_dict

Name	Score
Alice	97
Bob	76
John	79
Doe	27

Q2: การใช้งาน dict1 กับ list1 ด้านล่างแตกต่างกันหรือไม่

```
dict1 = {0:10, 1:20, 2:30, 3:40, 4:100}
list1 = [10, 20, 30, 40, 100]
```

Q3: จากข้อ Q1 ให้เพิ่ม นศ 1 คนที่มีเกรดและคะแนนดังข้างล่างใน grade_dict และ

score_dict

grade_dict

Name	Grade
Nor	A*

score_dict

Name	Score
Nor	999

Q4: จง print `grade_dict` และ `score_dict` ดัง pattern ด้านล่าง

Alice has grade A and score 97

Bob has grade B+ and score 76

John has grade B+ and score 79

Doe has grade F and score 27

Nor has grade A* and score 999

Q5: ให้สร้าง **tuple** เก็บข้อมูล (**Name, Grade, Score**) ของนักศึกษาแต่ละคนจาก Q4 เช่น ("Alice", "A", 97) แล้วเก็บ tuple ทั้งหมดไว้ใน **list of tuple** ชื่อ `student_tuple_list`

Q6: จาก `student_tuple_list` ให้เขียนโค้ดเพื่อ:

1. print รายชื่อ (Name) นักศึกษาทั้งหมด
 2. print คะแนนสูงสุด และชื่อนักศึกษาที่ได้คะแนนนั้น
-

Q7: จงสร้าง **set** ของเกรดนักศึกษาทั้งหมดจาก `grade_dict` เช่น {"A", "B+", "F", "A*"}

Q8: สมมติว่ามีอีก set ของเกรดที่ถือว่า ผ่าน (passing grades)

```
passing = {"A", "A*", "B+", "B"}
```

ให้หาว่าใน `grade_set` ของ Q7 มีนักศึกษากี่คนที่ได้เกรด ไม่ผ่าน

Q9: ถ้ามี set ของรายชื่อนักศึกษาที่ลงทะเบียนเรียน (enrolled students)

```
enrolled = {"Alice", "Bob", "John", "Doe", "Nor", "Sara"}
```

ให้หาว่าใครบ้างที่ ลงทะเบียน แต่ ไม่มีคะแนนใน `score_dict`

Q10: ให้ข้อมูล `grade_dict` (ชื่อ → เกรด) ให้พิมพ์จำนวนคนต่อแต่ละเกรด และเรียงลำดับตามตัวอักษร

Input

```
grade_dict = {  
    "Alice": "A",
```

```
"Bob": "B+",  
"John": "B+",  
"Doe": "F",  
"Nor": "A*",  
"Nina": "A",  
"Mark": "B"  
}
```

Expected Output

```
A : 2  
A* : 1  
B : 1  
B+ : 2  
F : 1
```

Q11: รวม `grade_dict` และ `score_dict` ให้เป็น `profile_dict` → (grade, score)

- ถ้าไม่มีเกรด ให้เป็น "N/A"
- ถ้าไม่มีคะแนน ให้เป็น -1

Input

```
grade_dict = {"Alice": "A", "Bob": "B+", "John": "B+", "Doe": "F", "Nor": "A*"}  
score_dict = {"Alice": 97, "Bob": 76, "John": 79, "Doe": 27, "Sara": 88}
```

Expected Output

```
Alice -> (A, 97)  
Bob -> (B+, 76)  
Doe -> (F, 27)  
John -> (B+, 79)  
Nor -> (A*, -1)  
Sara -> (N/A, 88)
```

Q12: หา 3 คนที่ดีที่สุด โดยเรียงตาม

1. คะแนนมาก → น้อย
2. เกรดน้ำหนัก ($A^* > A > B+ > B > C > D > F$)

Input

```
grade_dict = {"Alice": "A", "Bob": "B+", "John": "B+", "Doe": "F", "Nor": "A*", "Nina": "A"}  
score_dict = {"Alice": 97, "Bob": 92, "John": 92, "Doe": 27, "Nor": 92, "Nina": 95}  
k = 3
```

Expected Output

Alice A 97
Nina A 95
Nor A* 92

Q13: สร้าง `student_tuples (name, grade, score)` และกรองนักศึกษาที่คะแนน ≥ 90 และเกรด $\neq F$

Input

```
grade_dict = {"Alice": "A", "Bob": "B+", "John": "B+", "Doe": "F", "Nor": "A*", "Nina": "A", "Max": "B"}  
score_dict = {"Alice": 97, "Bob": 92, "John": 89, "Doe": 27, "Nor": 95, "Nina": 90, "Max": 91}
```

Expected Output

`[('Alice', 'A', 97), ('Nor', 'A*', 95), ('Nina', 'A', 90), ('Bob', 'B+', 92), ('Max', 'B', 91)]`

Q14: คำนวณค่าเฉลี่ยคะแนนต่อเกรด และหาคนที่คะแนนสูงสุดในแต่ละเกรด

Input

```
student_tuples = [  
    ("Alice", "A", 97),  
    ("Nor", "A*", 95),  
    ("Nina", "A", 90),  
    ("Bob", "B+", 92),  
    ("Max", "B", 91),  
    ("John", "B+", 89)  
]
```

Expected Output

Avg by grade:
A* : 95.00
A : 93.50
B+ : 90.50
B : 91.00

Top by grade:
A* : Nor(95)

A : Alice(97)
B+ : Bob(92)
B : Max(91)

Q15: หาวว่าใครบ้างที่

- ลงทะเบียนแต่ไม่มีคะแนน
- มีคะแนนแต่ไม่ได้ลงทะเบียน

Input

```
enrolled = {"Alice", "Bob", "John", "Doe", "Nor", "Sara", "Max"}  
score_dict = {"Alice": 97, "Bob": 76, "John": 79, "Doe": 27, "Nina": 90}
```

Expected Output

Enrolled but no score: {'Nor', 'Sara', 'Max'}
Scored but not enrolled: {'Nina'}

Q16: หาเกรดที่ผิดรูปแบบ และเกรดที่ยังไม่มีใครได้

Input

```
possible_grades = {"A*", "A", "B+", "B", "C", "D", "F"}  
grade_dict = {"Alice": "A", "Bob": "B+", "John": "B+", "Doe": "F", "Nor": "A*", "Nina": "A",  
"Max": "B", "Weird": "E"}
```

Expected Output

Invalid grades in data: {'E'}
Unused possible grades: {'C', 'D'}

Q17: จงเขียนโปรแกรมเพื่อสร้างและจัดอันดับข้อมูลนักศึกษาตามเงื่อนไขต่อไปนี้:

1. นำข้อมูลจาก **grade_dict** และ **score_dict** มาสร้าง list ของ tuple รูปแบบ (name, grade, score, passed_bool)
 - หากไม่มีคะแนน ให้ไม่รวมคนนั้นในการจัดอันดับ
 - **passed_bool** เป็น True ถ้าเกรดอยู่ใน passing set, มิฉะนั้นเป็น False.
2. จัดอันดับ (Ranking)
เรียงลำดับ tuple ตามเงื่อนไข:
 - คนที่ **passed_bool=True** มาก่อน False
 - คะแนนจากมาก → น้อย

- หากคะแนนเท่ากัน ให้เรียงตาม “น้ำหนักของเกรด” ตามลำดับนี้:
A* > A > B+ > B > C > D > F
- ถ้ายังเท่ากัน ให้เรียงตามชื่อในลำดับตัวอักษร (A → Z)

3. แสดงผลลัพธ์

- พิมพ์ Ranking ตามรูปแบบ
Name Grade Score PASS/FAIL
- สรุปท้ายผลลัพธ์:
 - จำนวนคนที่ผ่าน (Passed)
 - จำนวนคนที่ไม่ผ่าน (Failed)
 - รายชื่อ Top-3 ของคนที่ผ่าน (เรียงตามอันดับที่ได้)
 - เกรดที่ “ยังไม่มีใครได้” (Unused grades) โดยเปรียบเทียบกับ possible_grades

Input

```
grade_dict = {
    "Alice": "A",
    "Bob": "B+",
    "John": "B+",
    "Doe": "F",
    "Nor": "A*",
    "Nina": "A",
    "Max": "B"
}
```

```
score_dict = {
    "Alice": 97,
    "Bob": 92,
    "John": 92,
    "Doe": 27,
    "Nor": 95,
    "Nina": 90
}
```

```
possible_grades = {"A*", "A", "B+", "B", "C", "D", "F"}
passing = {"A*", "A", "B+", "B"}
```

Expected Output

Ranking:

```
Alice A 97 PASS
Nor A* 95 PASS
Bob B+ 92 PASS
John B+ 92 PASS
```

Nina A 90 PASS

Doe F 27 FAIL

Summary:

Passed: 5

Failed: 1

Top-3 passed: ['Alice', 'Nor', 'Bob']

Unused grades: {'C', 'D'}

Q18: ลบข้อมูลซ้ำและสร้างตารางข้อมูลเรียงตามชื่อ

กำหนดข้อมูลนักศึกษาเป็น **list ของ tuple** ในรูปแบบ
(name, grade, score)

จงเขียนโปรแกรมเพื่อ

1. ตรวจสอบว่ามีชื่อใดปรากฏมากกว่า 1 ครั้งหรือไม่ (ชื่อซ้ำ)
2. หากชื่อซ้ำ ให้เก็บเฉพาะ record ที่มี **score สูงสุด** ของชื่อนั้น (ถ้าคะแนนเท่ากัน ให้เก็บอันแรก
ที่พบ)
3. เก็บชื่อที่ถูกลบออกเนื่องจากซ้ำไว้ใน **dropped_duplicates** (เป็น set)
4. สร้าง **list ของ tuple** รูปแบบ
(id, name, grade, score)
โดย id เริ่มจาก 1 และเรียงลำดับตามตัวอักษรของชื่อ ($A \rightarrow Z$)
5. แสดงผล list และ dropped set

Input

```
raw_data = [  
    ("Alice", "A", 97),  
    ("Bob", "B+", 76),  
    ("John", "B+", 79),  
    ("Doe", "F", 27),  
    ("Nor", "A*", 92),  
    ("Alice", "A", 95)    # ชื่อซ้ำ  
]
```

Expected Output

Rows:

```
(1, 'Alice', 'A', 97)  
(2, 'Bob', 'B+', 76)  
(3, 'Doe', 'F', 27)  
(4, 'John', 'B+', 79)  
(5, 'Nor', 'A*', 92)
```

Dropped duplicates: {'Alice'}